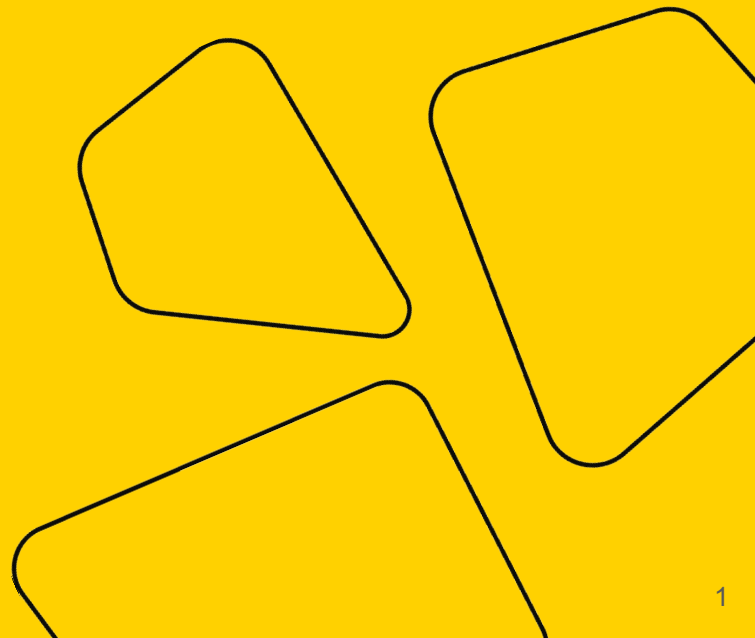
Machine Learning Lecture 7: kNN indexes and hyperparameter optimization

Iurii Efimov

girafe ai



Outline

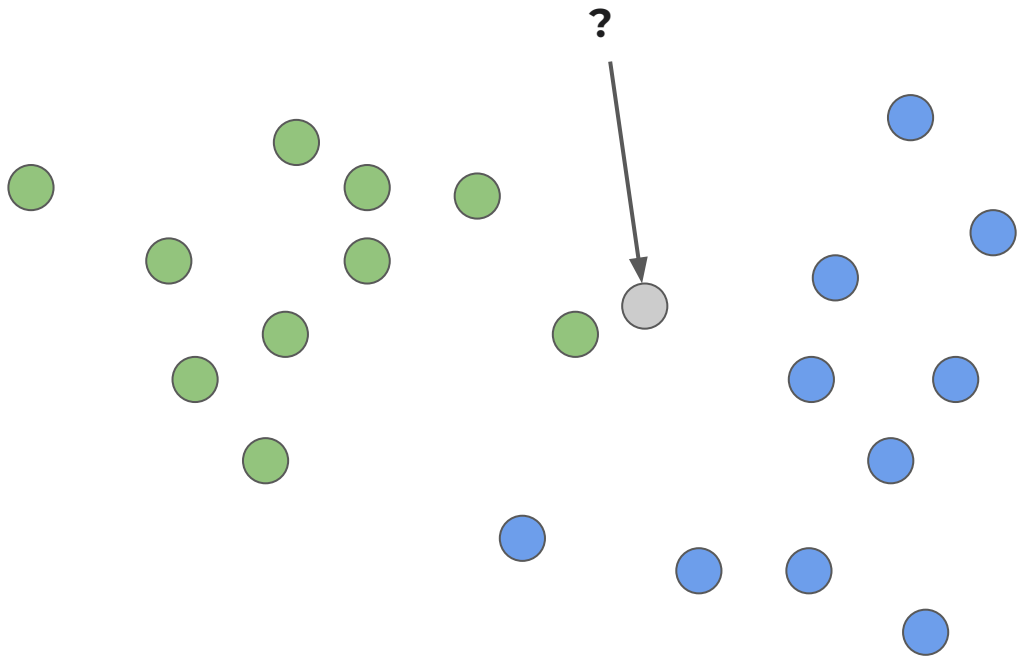
1. kNN
2. Indexes
3. Feature importances
4. Hyperparameter optimization

k Nearest Neighbors

girafe
ai

01

Intuition



kNN algorithm

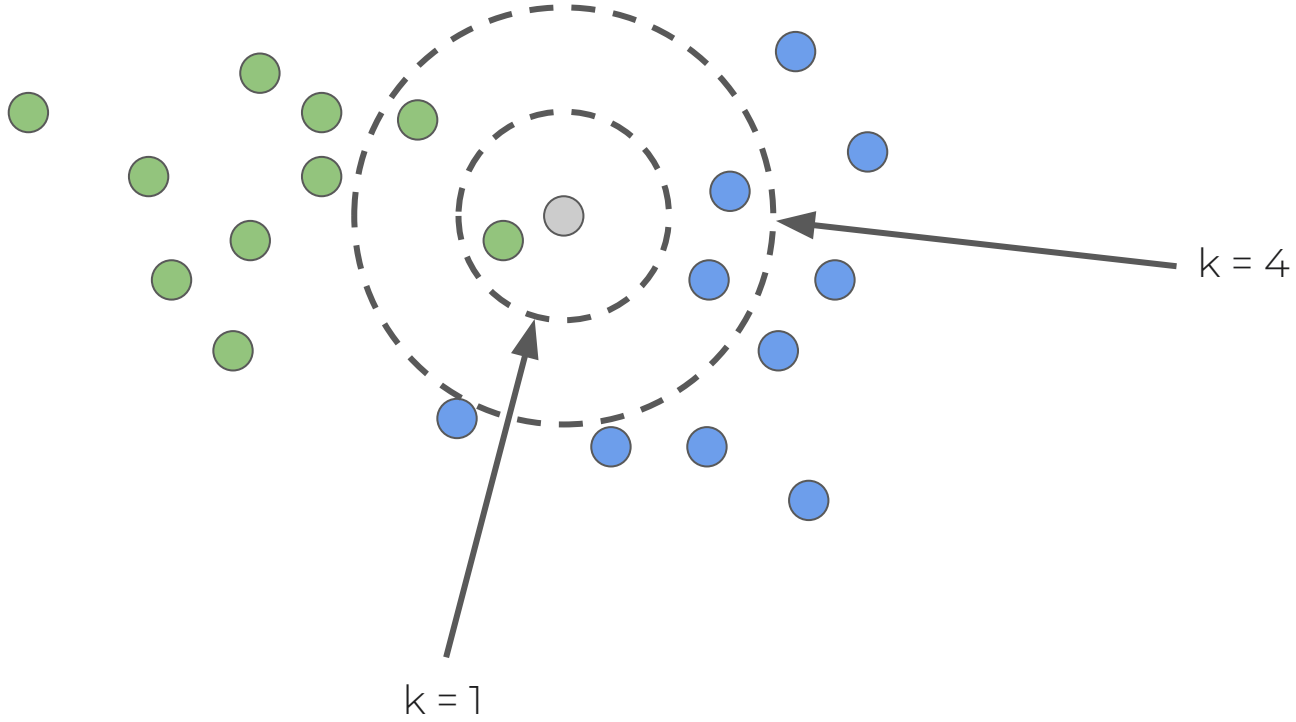
Given a new observation:

1. Calculate the distance to each of the samples in the dataset
2. Select samples from the dataset with the minimal distance to them
3. The label of the new observation will be the most frequent label among those nearest neighbors

How to answer to regression problem?

How to make it better?

1. The number of neighbors k



How to make it better?

1. The number of neighbors k
2. The distance measure between samples
    a. Euclidean (L2)
    b. city block distance (aka Manhattan, L1)
    c. Minkowski distances
    d. cosine
    e. Hamming
        i. number of substitutions
    f. etc
3. Weighted neighbours
    a. fake! (never used in practice)

They are hyperparameters for kNN model

Parameters and hyperparameters

Parameters:

none. This is non-parametric model.

Hyperparameters:

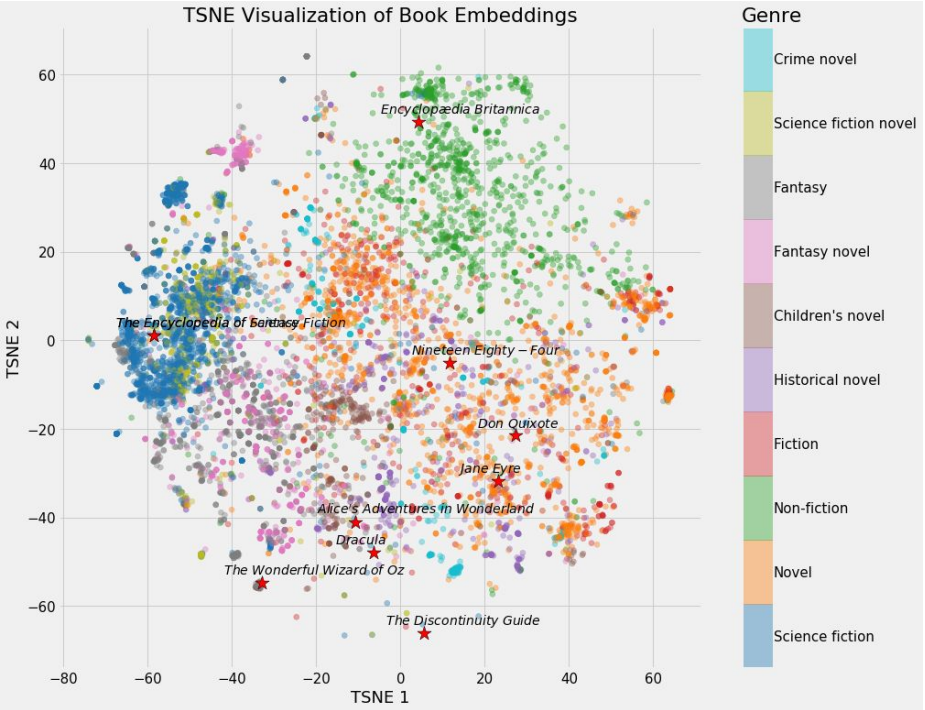
- k - number of neighbours
- distance function
  - euclidean, cosine distance

How kNN is used?

Main object to apply kNN nowadays are embeddings (vectors) produced by neural networks.

Such embeddings represent object properties in a compact format.

Nearest neighbors are used to roughly select possible candidates for further processing.



Usage examples

- Information retrieval problems
  - search engines
  - recommender systems
  - RAG

kNN indexes

girafe ai

02

Naive compute approach

Common object for kNN nowadays is embedding (usually by neural networks).

Let's suppose we have **million (10^6)** of floating point **embeddings in 100 dimensional** space.

We have a computer with **1 GHz (10^9) operations** frequency.

Then we want to find **10 nearest neighbours** of a new given embedding.

How much time will it take?

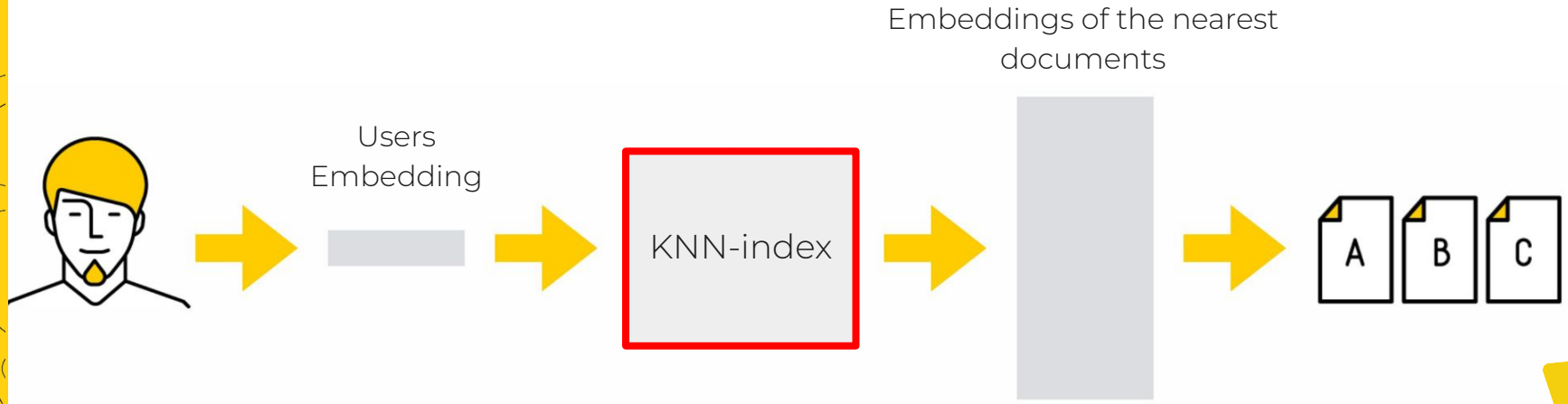
Is this time acceptable for real time operation?

kNN index

KNN index is a data structure
that allows you to quickly find the N nearest vectors for a given vector.

We perform some precompute beforehand to increase speed of final search.

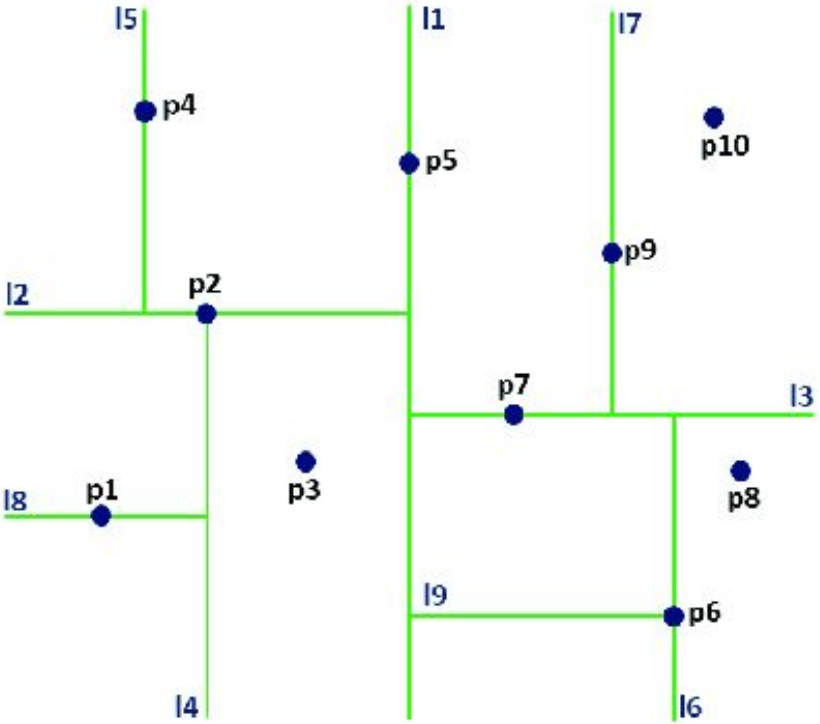
This approach is also called **Approximate Nearest Neighbor (ANN)**

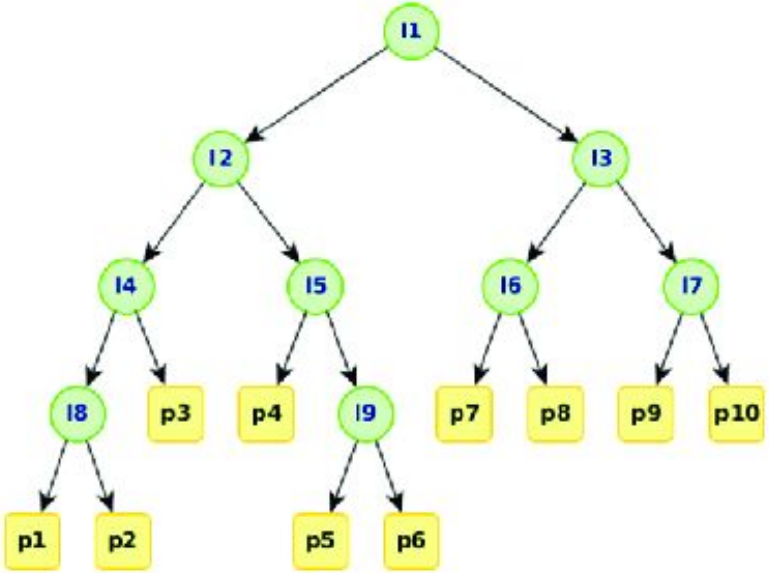


kNN index ideas

1. Dimensionality reduction
2. Clustering
3. Trees
    a. k-d tree
        i. default for scikit learn
        ii. ball tree is extension for high dimensional cases
4. Grid / Graph
    a. HNSW
        i. most common nowadays
5. many more...

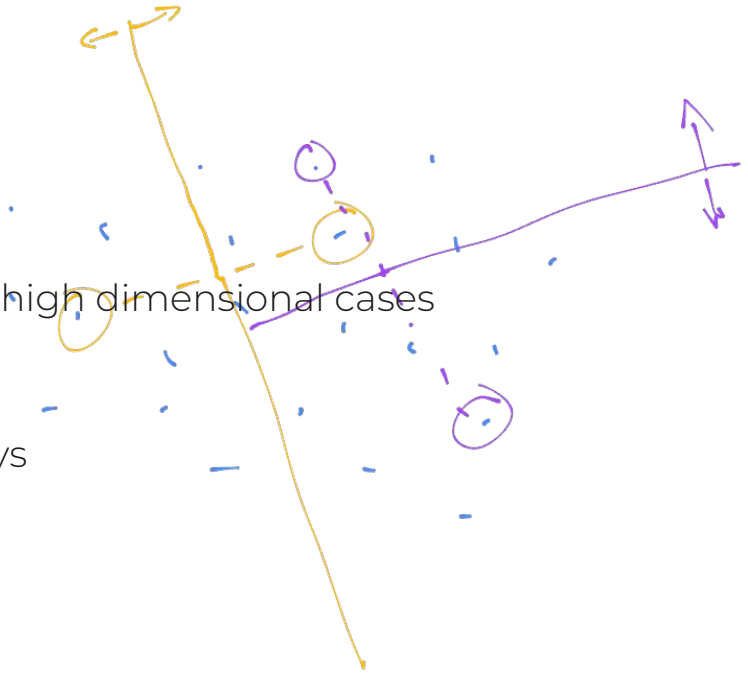
k-d tree


l5
l1
l7
p4
p10
p5
p9
l2
p2
p7
l3
p3
p8
l8
p1
l9
p6
l4
l6


l1
l2
l3
l4
l5
l6
l7
l8
p3
p4
l9
p7
p8
p9
p10
p1
p2
p5
p6

Approximate Nearest Neighbor Oh Yeah (ANNOY) by Spotify

1. Dimensionality reduction
2. Clustering
3. Trees
    a. k-d tree
        i. default for scikit learn
        ii. ball tree is extension for high dimensional cases
4. Grid / Graph
    a. HNSW
        i. most common nowadays
5. many more...



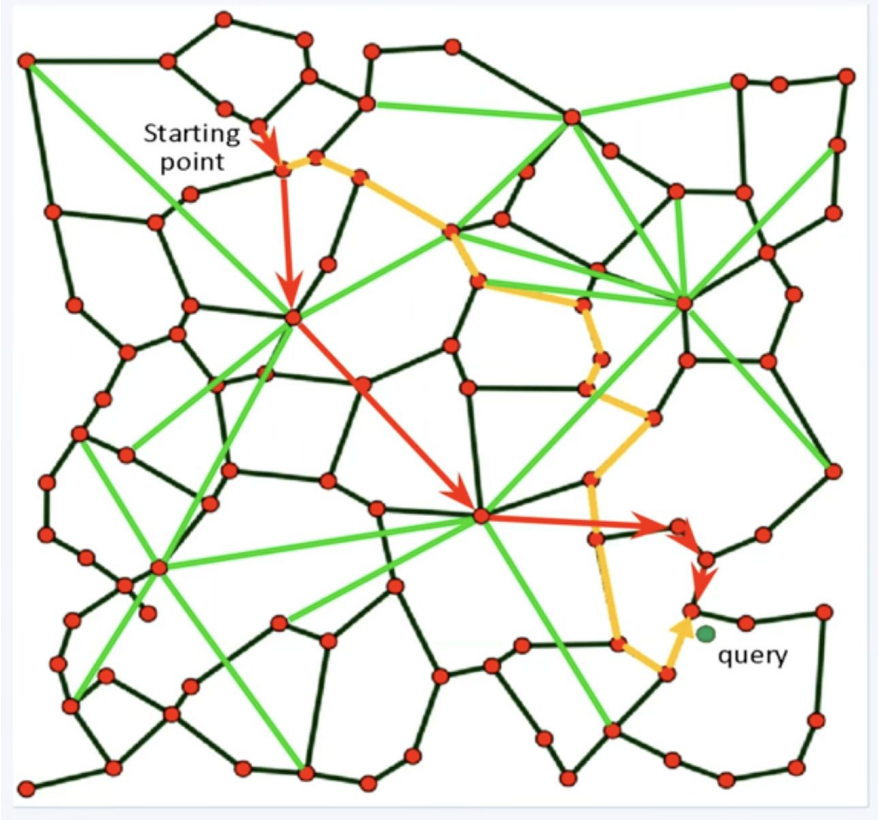
Navigable Small World

- A graph where vertices are embeddings, edges are distances
- Adding nearby and remote vertices as edges
- We start from a random vertex, count the distances and move along the graph closer to the query point

https://arxiv.org/abs/1603.09320

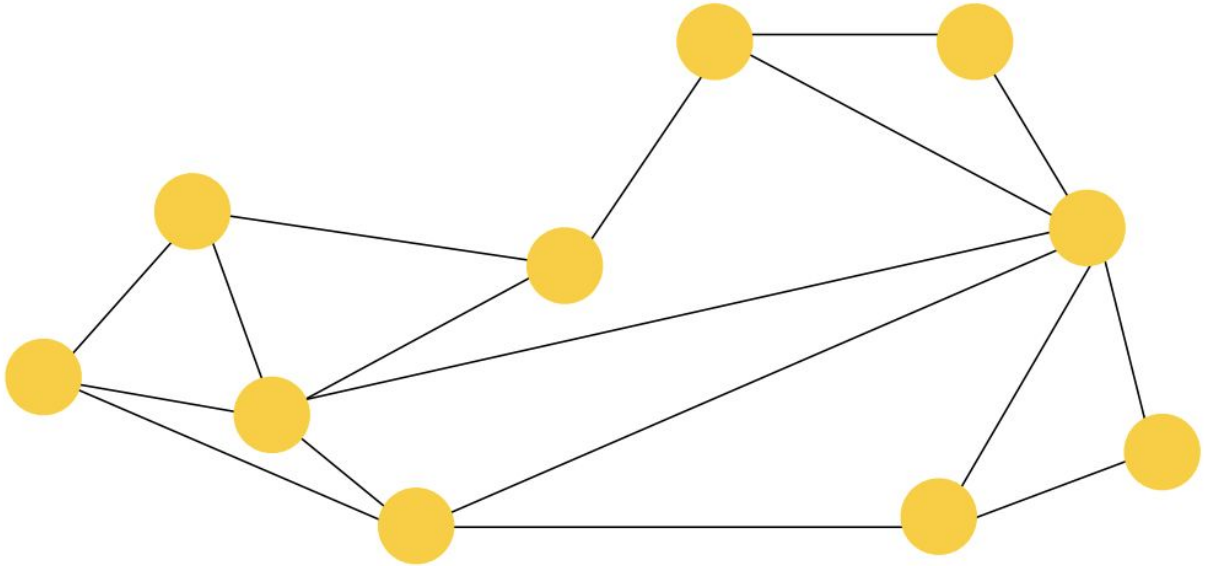
NSW (Navigable Small World)

https://www.youtube.com/watch?v=4PsyNdFlxmk

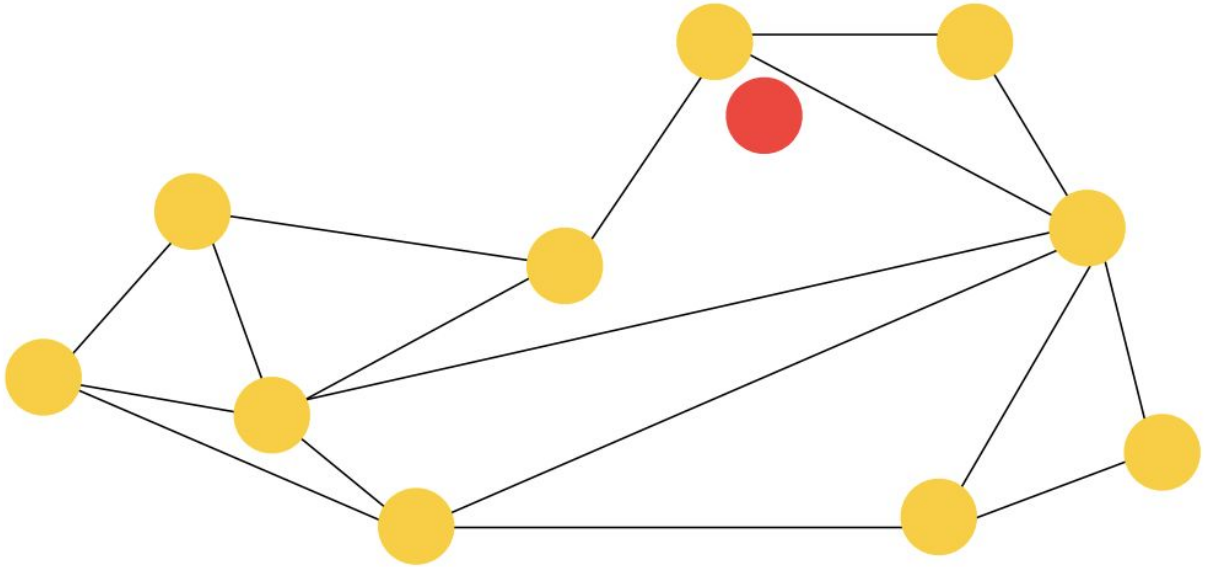


1. Connect nearest neighbours
2. Add long connections

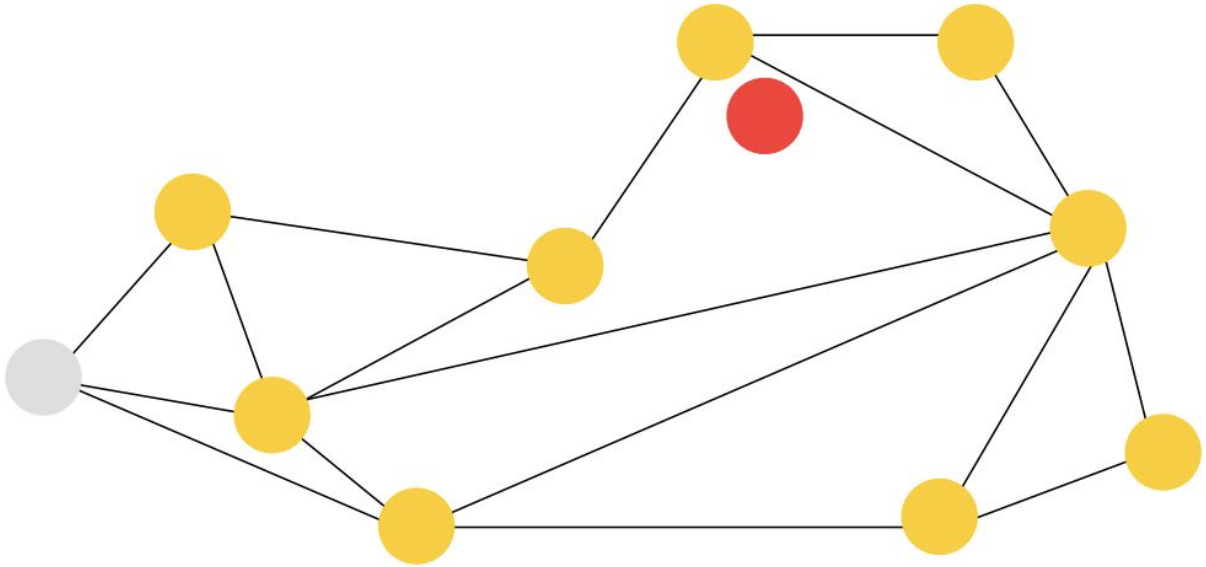
NSW (Navigable Small World)



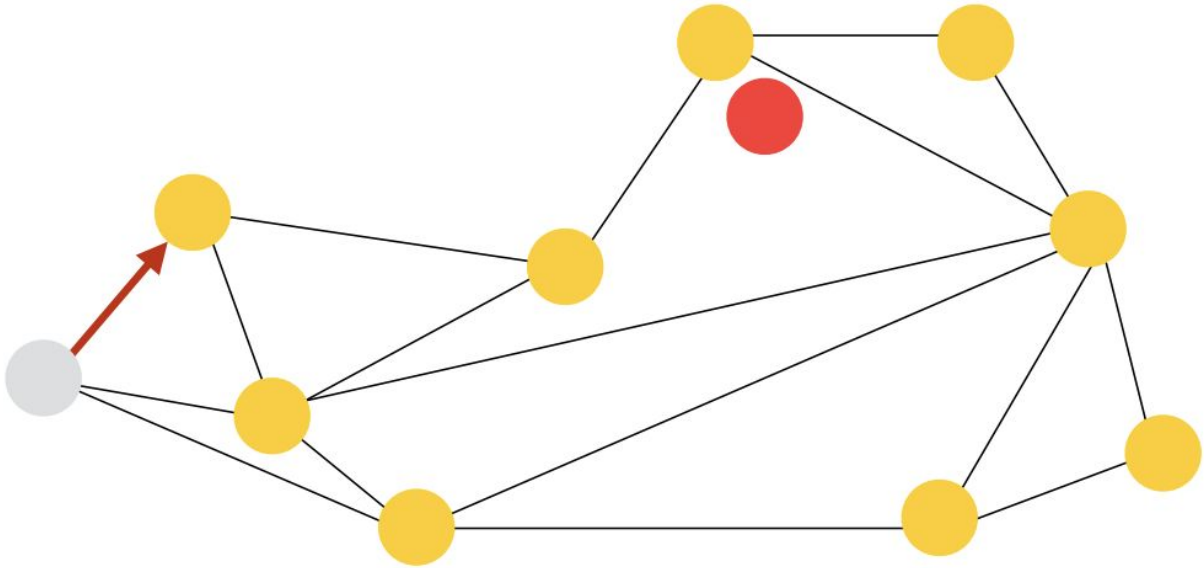
NSW (Navigable Small World)



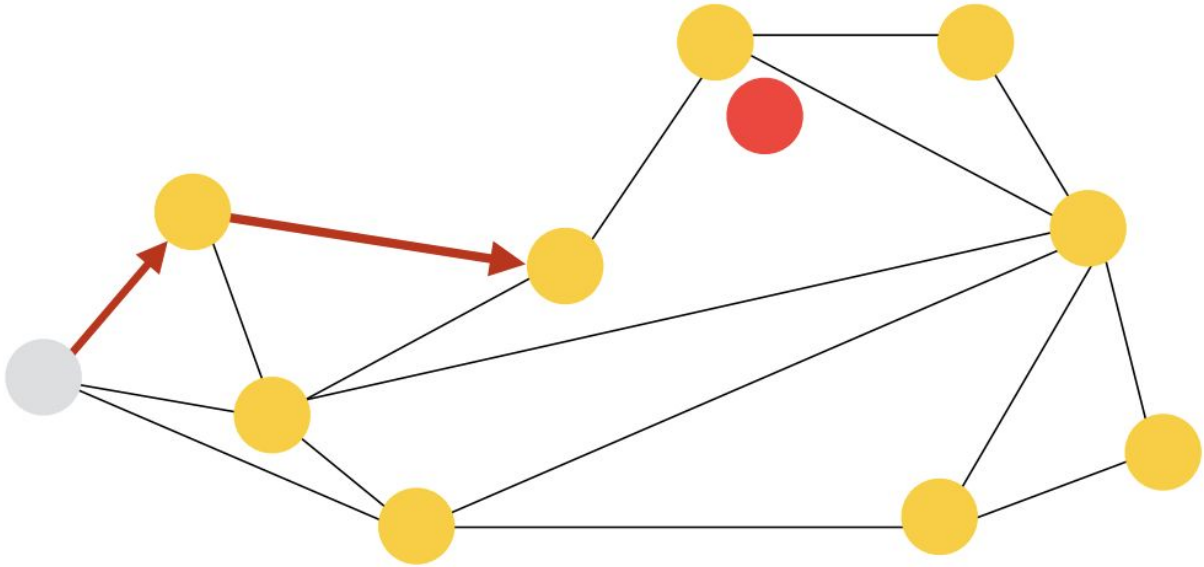
NSW (Navigable Small World)



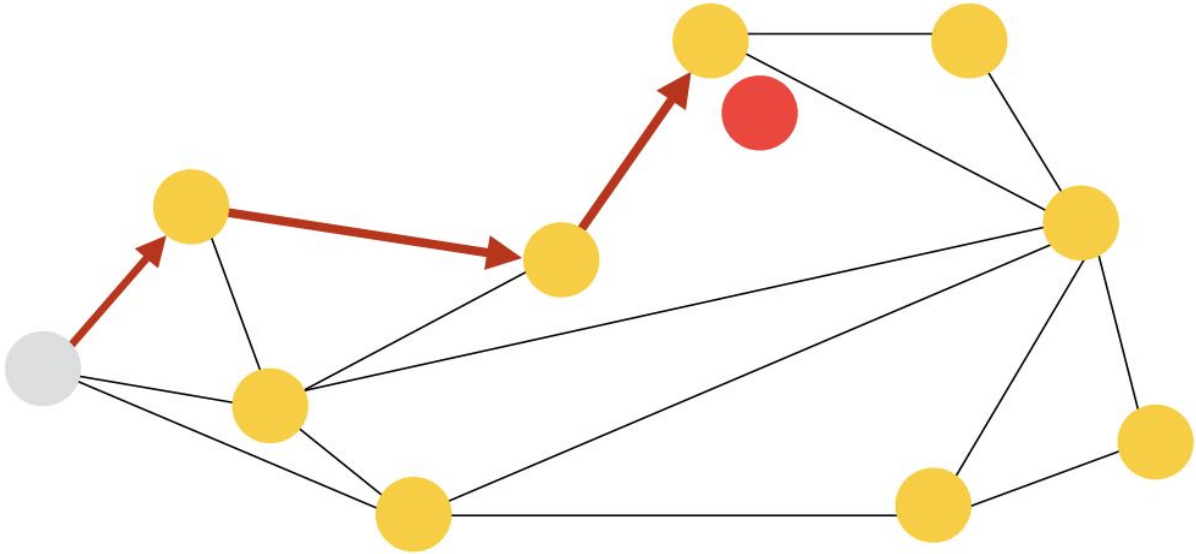
NSW (Navigable Small World)



NSW (Navigable Small World)



NSW (Navigable Small World)

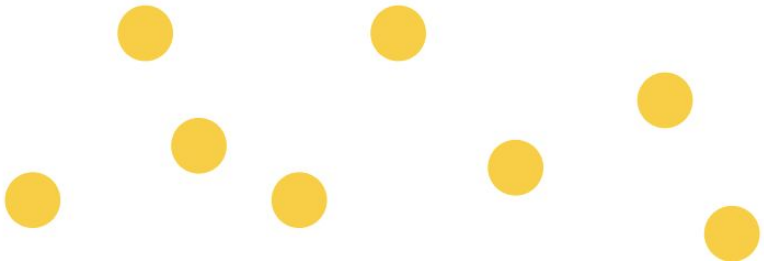


Hierarchical Navigable Small World (HNSW)

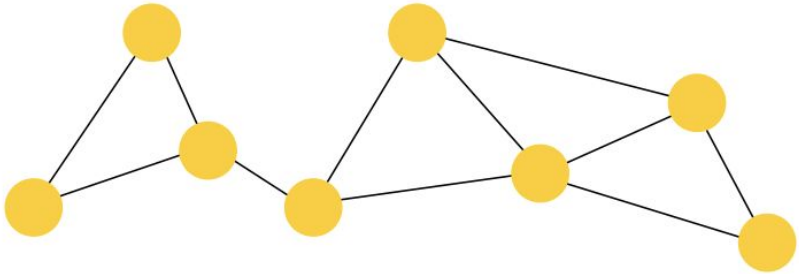
girafe ai

03

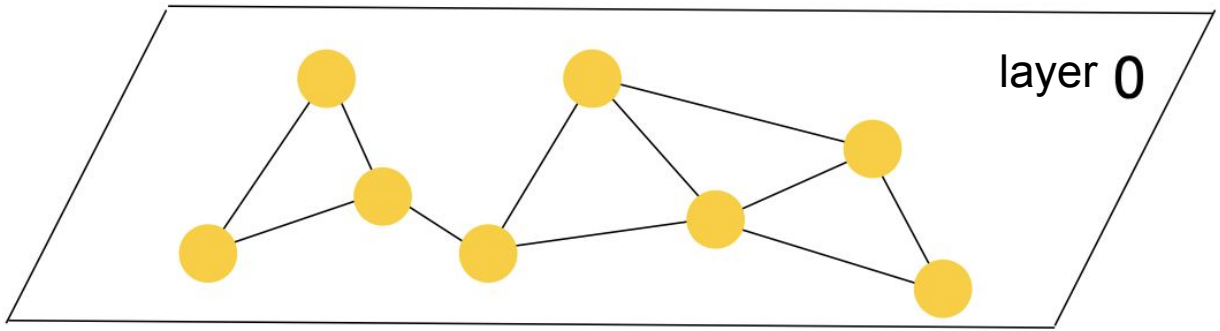
HNSW



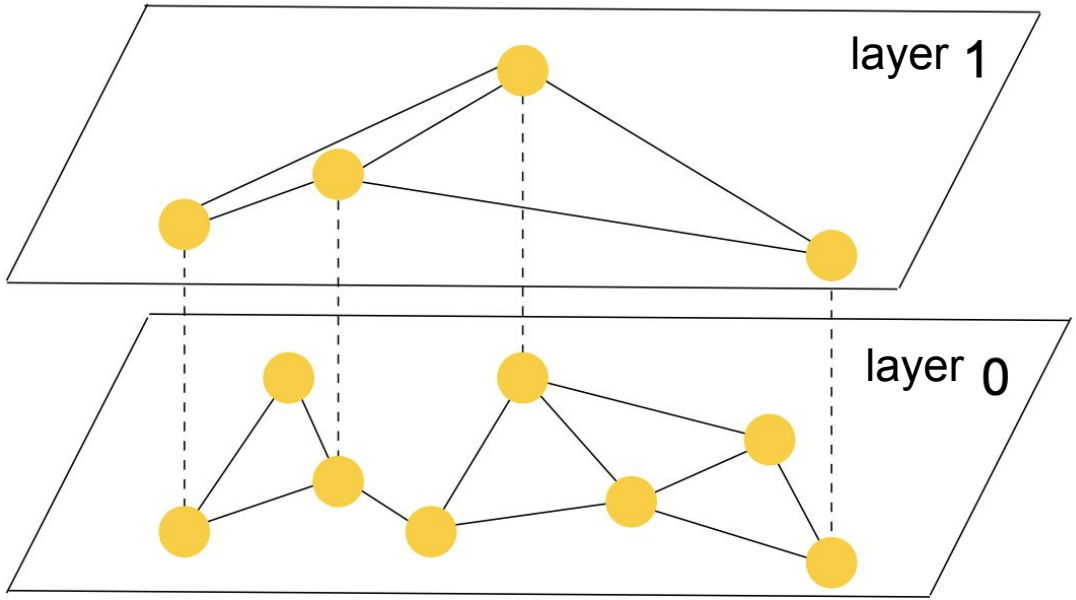
HNSW



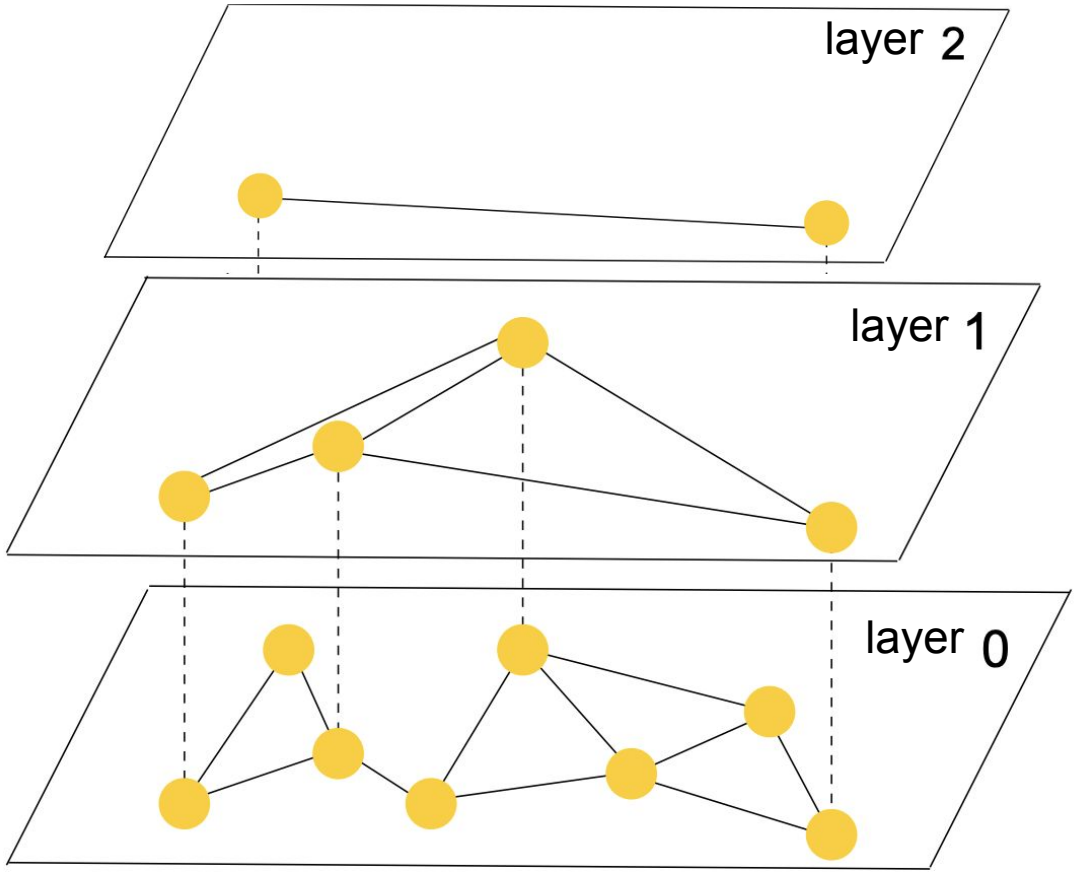
HNSW



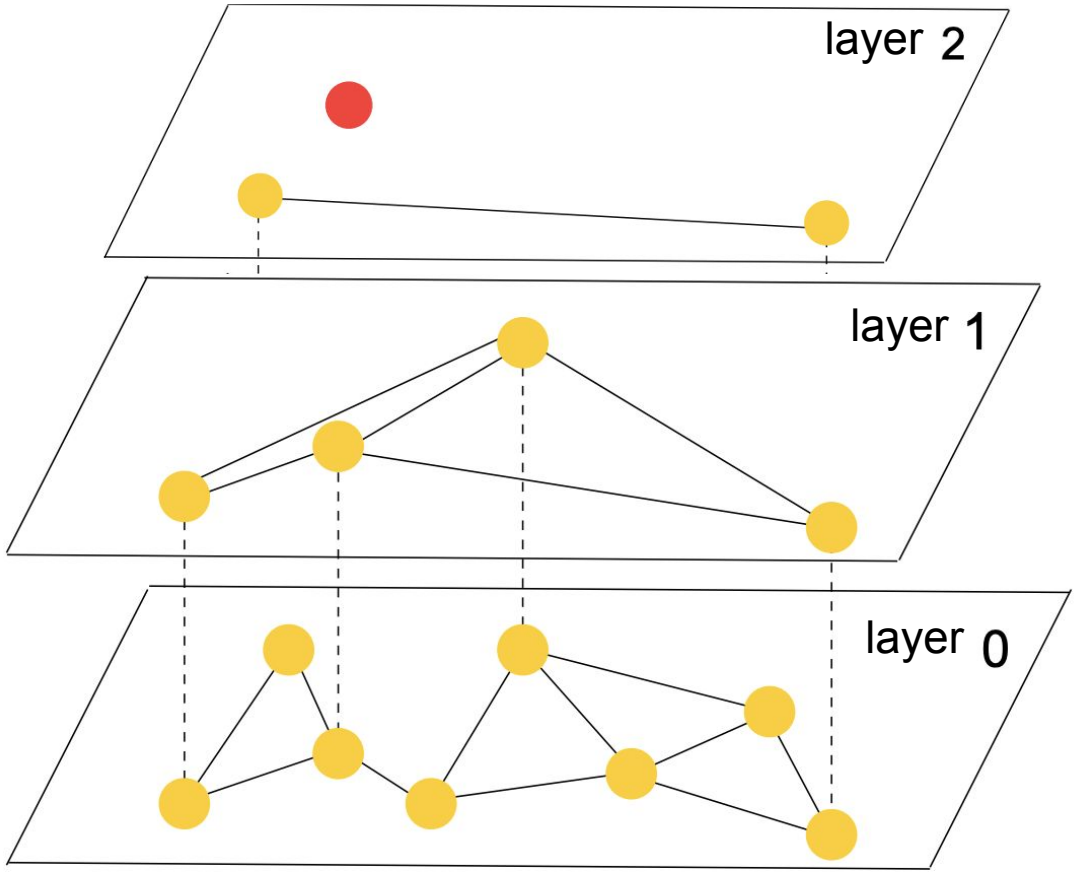
HNSW



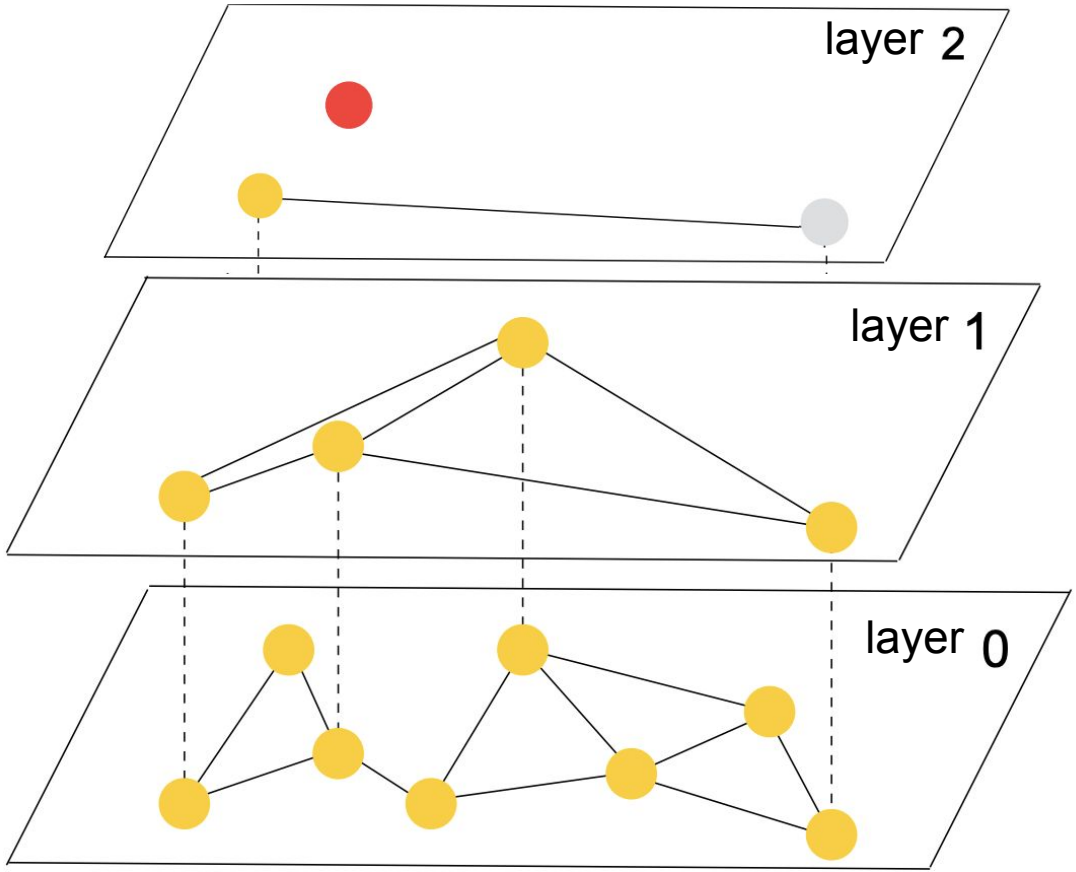
HNSW



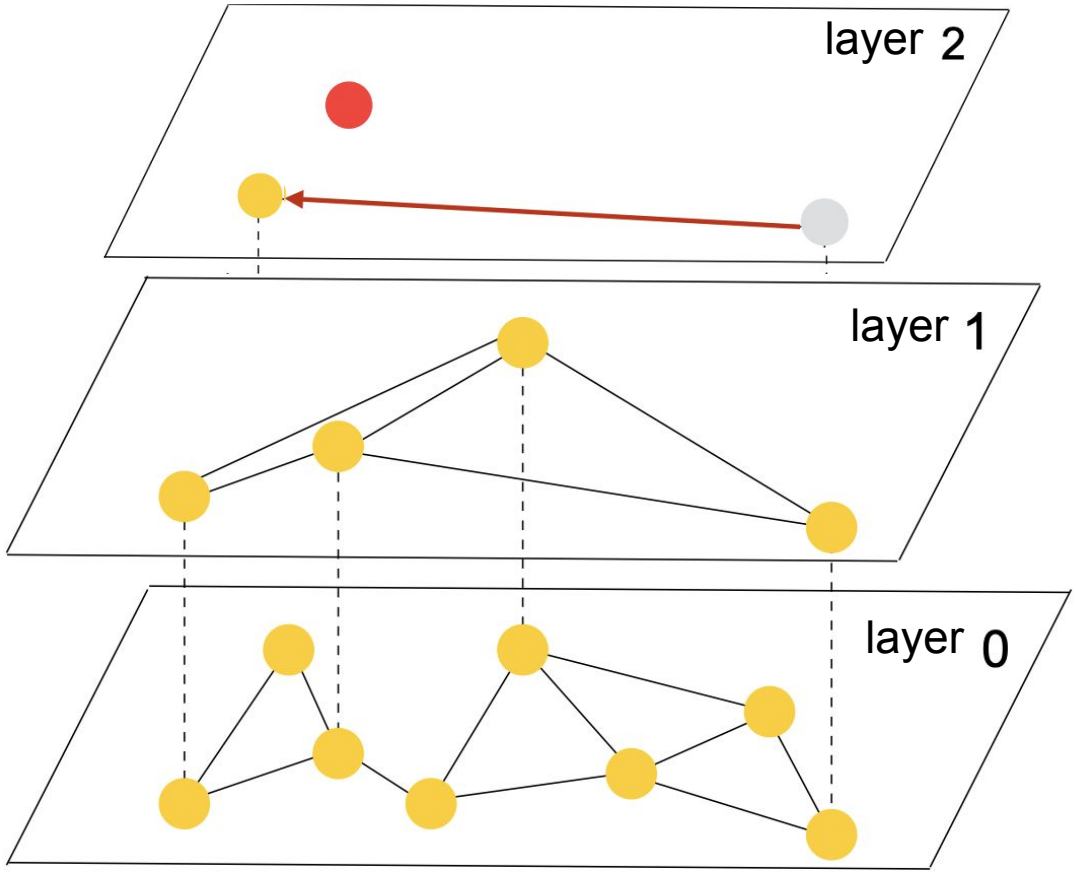
HNSW



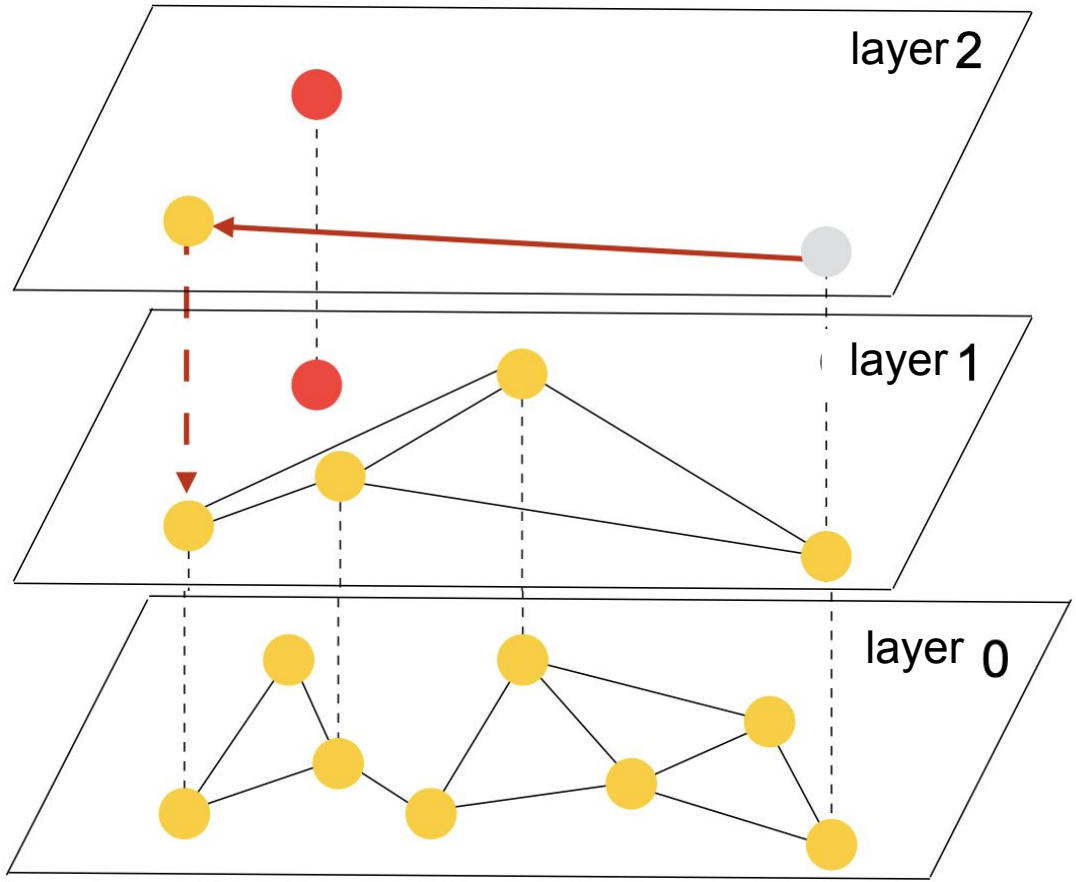
HNSW



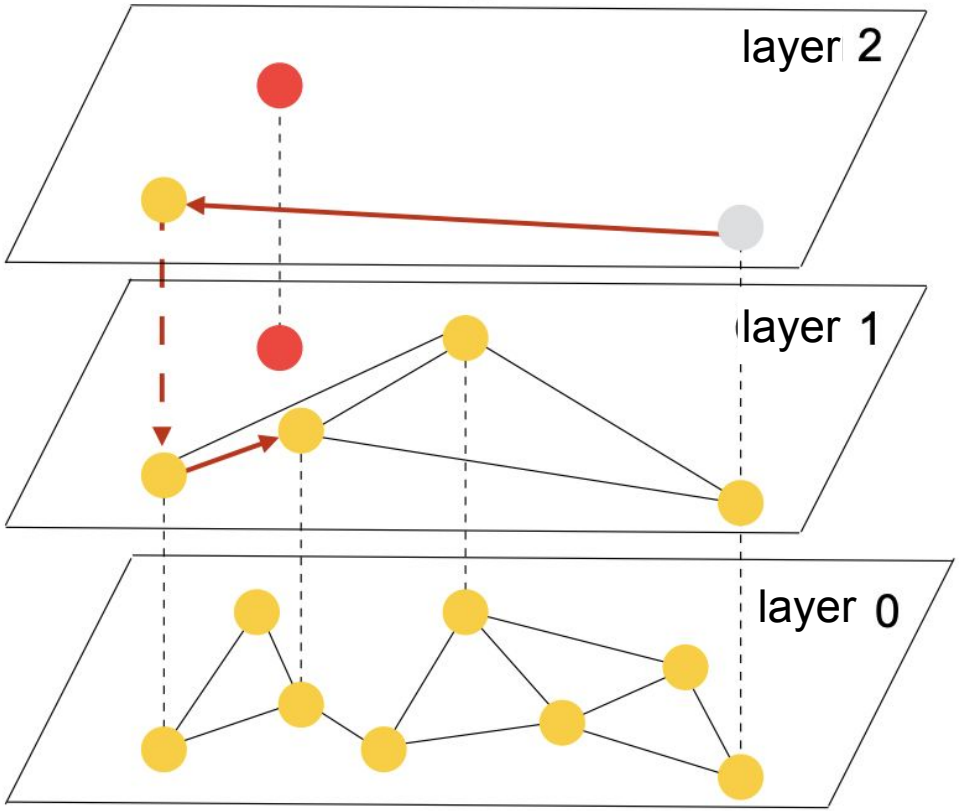
HNSW



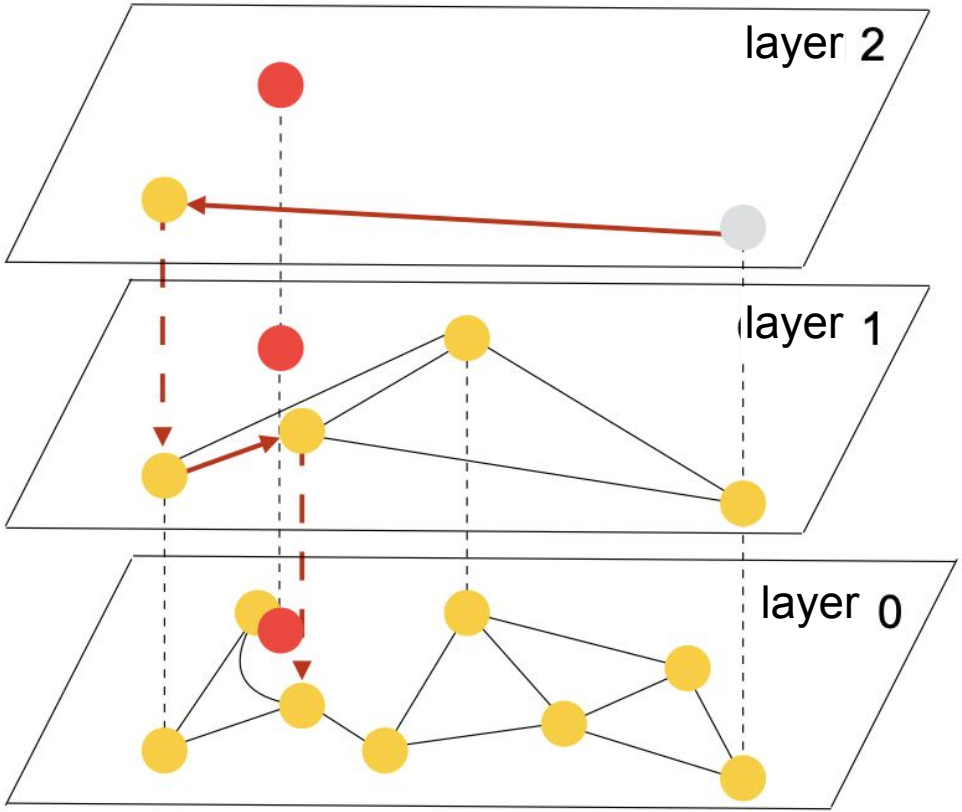
HNSW



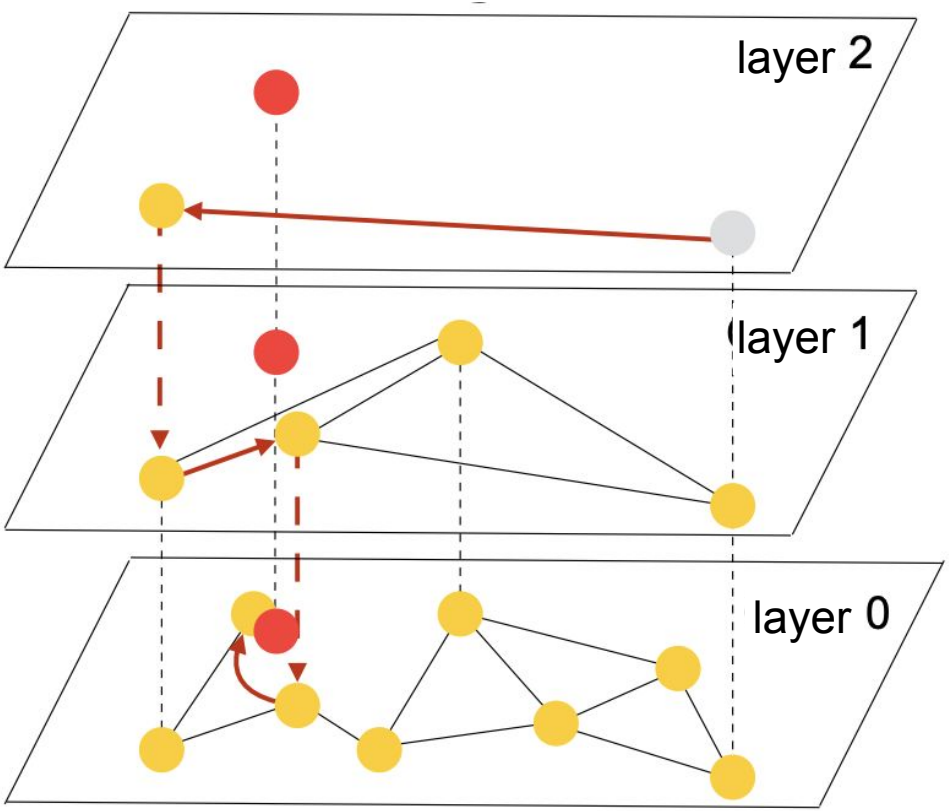
HNSW



HNSW

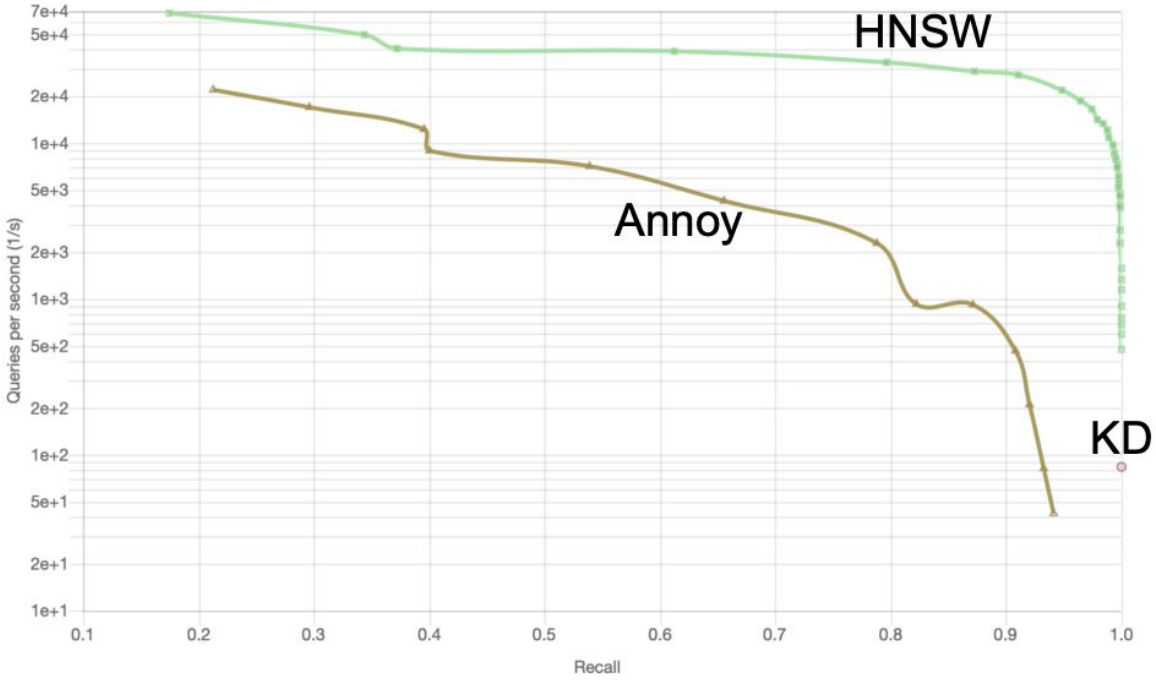


HNSW



Accuracy of approximate search

- In addition to speed, you need to find accurate nearby objects
- Comparison on ALS embedding last fm-64-dot, 360K



The bitter lesson

Rich Sutton, author of many Reinforcement Learning algorithms, wrote an article in 2019.

“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin”

Methods have to be scalable such that:

- benefit from more compute
- benefit from more data

otherwise they will be discarded by Moore’s law in a very short term.

Original text, Wikipedia

kNN index implementations

- https://github.com/facebookresearch/faiss
  - https://habr.com/ru/companies/okkamgroup/articles/509204/
- https://github.com/nmslib/nmslib
- https://github.com/flann-lib/flann
- https://github.com/spotify/annoy
- https://lucene.apache.org/core/9_1_0/core/org/apache/lucene/util/hnsw/package-summary.html

Vector databases

Also list of vector databases from openAI
https://platform.openai.com/docs/guides/embeddings/how-can-i-retrieve-k-nearest-embedding-vectors-quickly

Ready to production solutions

- FOSS
  - https://github.com/qdrant/qdrant
    - Has helm chart + distributed deployment
  - https://github.com/feast-dev/feast
  - https://github.com/NVIDIA/aistore
- Proprietary
  - https://www.pinecone.io/
  - https://www.trychroma.com/

Hyperparameter optimization

girafe ai

04

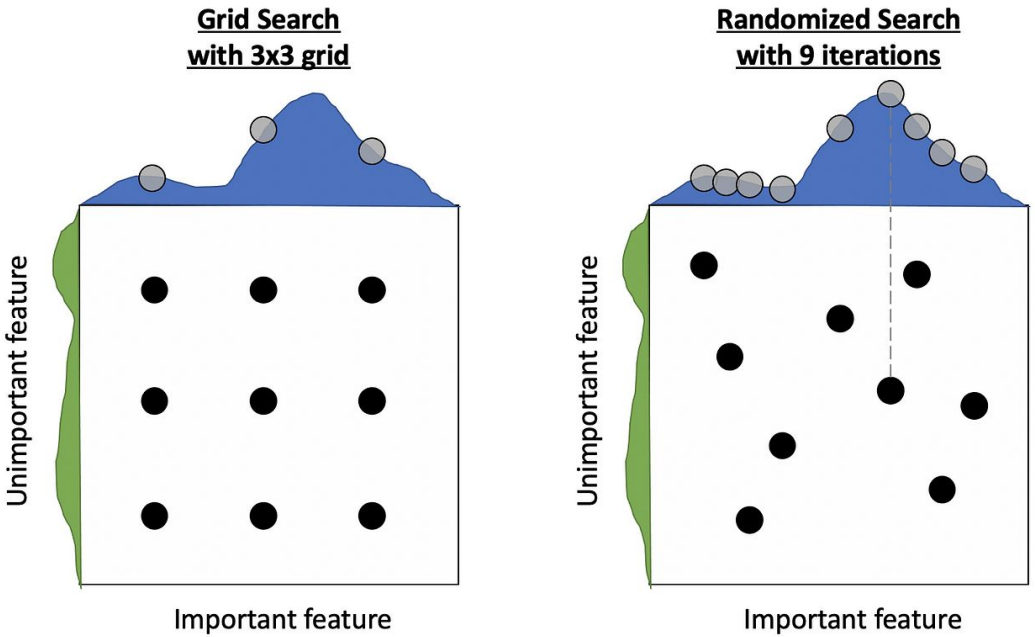
Optimization note

In optimization theory methods are associated with the order of derivatives they use. Main ones are:

- first order optimization (gradient based optimization)
  - use gradient of optimized function
  - e.g. SGD which we discussed
- second order optimization (Newton's method)
  - use Hessian matrix
  - they are quite slow
- zero order (black box optimization)
  - don't need gradient, only values of optimized function
  - that's what we are interested in today

0-order optimization approaches

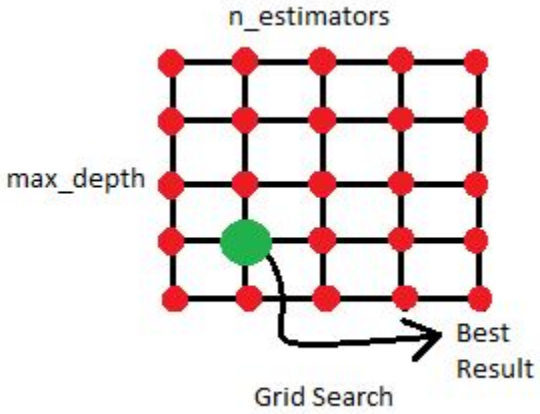
1. Manual trials
2. Grid search
3. Random search

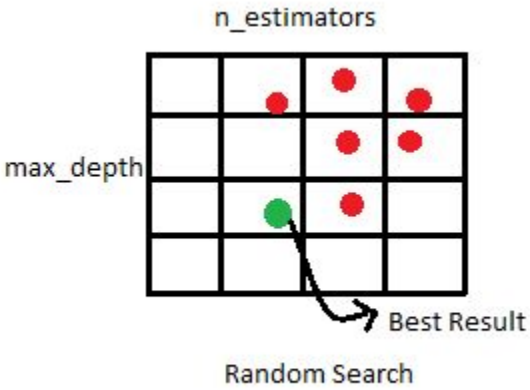


In theory

0-order optimization approaches

1. Manual trials
2. Grid search
3. Random search

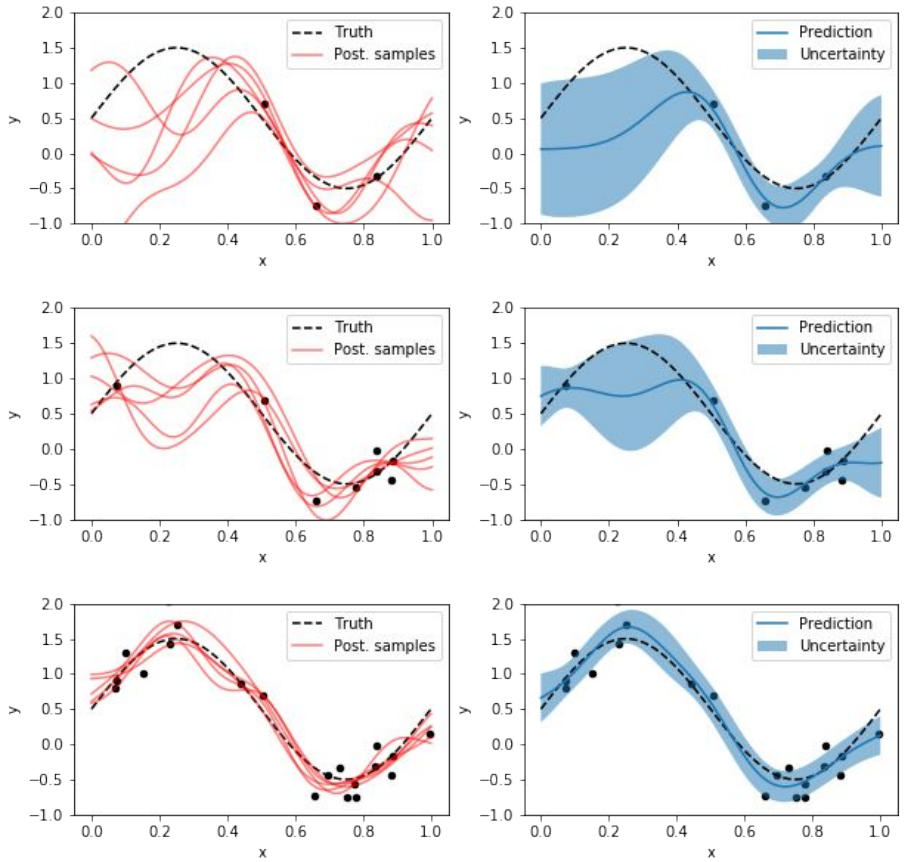




In practice

0-order optimization approaches

1. Manual trials
2. Grid search
3. Random search
4. Bayesian methods
5. Evolutionary methods



Main libraries

- Hyperopt
- Optuna
  - Documentation
  - Samplers list

OPTUNA

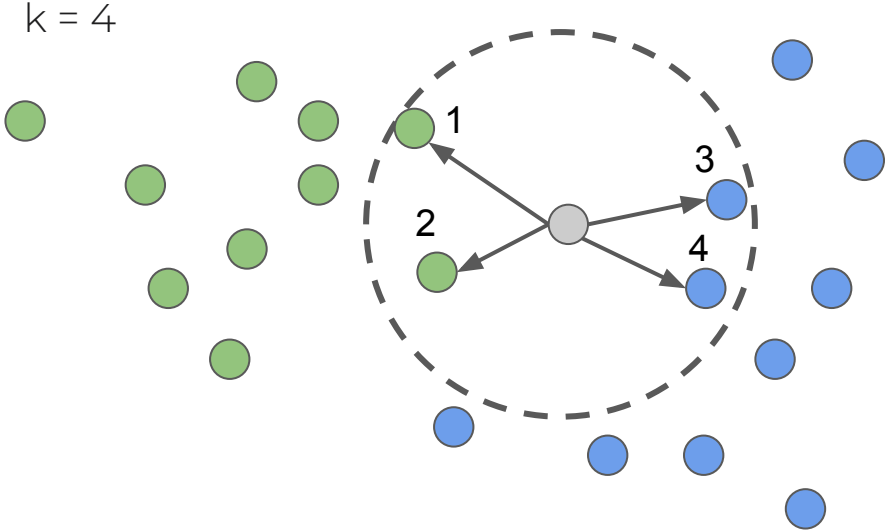
Revise

1. kNN
2. Indexes:
    a. KD Tree
    b. NSW / HSNW
3. Hyperparameter optimization

Thanks for attention!

Questions?

Weighted kNN



- Weights can be adjusted according to the neighbors order

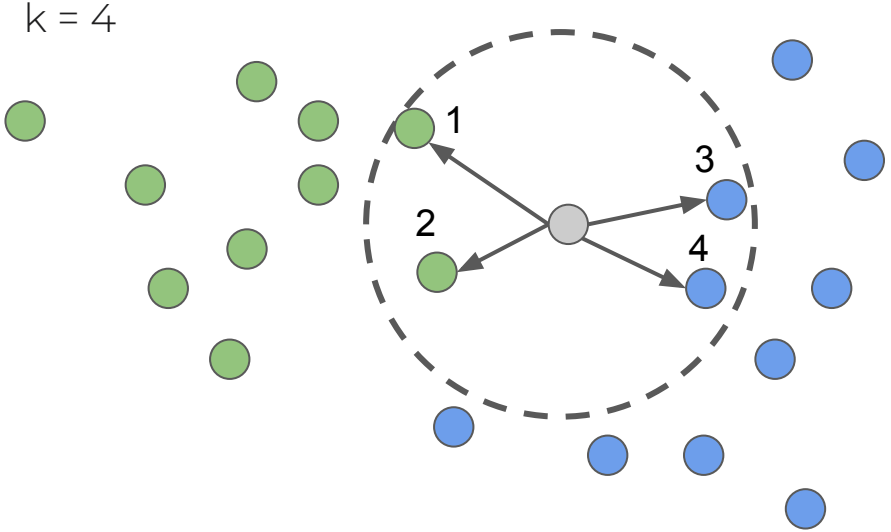
$$w(\mathbf{x}_{(i)}) = w_i$$

- or on the distance itself

$$w(\mathbf{x}_{(i)}) = w(d(\mathbf{x}, \mathbf{x}_{(i)}))$$

$$p_{\text{green}} = \frac{w(\mathbf{x}_1) + w(\mathbf{x}_2)}{w(\mathbf{x}_1) + w(\mathbf{x}_2) + w(\mathbf{x}_3) + w(\mathbf{x}_4)}$$

Weighted kNN



- Weights can be adjusted according to the neighbors order

$$w(\mathbf{x}_{(i)}) = w_i$$

- or on the distance itself

$$w(\mathbf{x}_{(i)}) = w(d(\mathbf{x}, \mathbf{x}_{(i)}))$$

$$p_{\text{blue}} = \frac{w(\mathbf{x}_3) + w(\mathbf{x}_4)}{w(\mathbf{x}_1) + w(\mathbf{x}_2) + w(\mathbf{x}_3) + w(\mathbf{x}_4)}$$